HMACSHA256 Auth

Authentication for iyzico services aligns with Basic Auth principles, while enhancing security through a precise sequence of encryption techniques, including PKI string, Base64, HMAC and SHA-256 hashing

To access our API securely, authentication is required. Authentication is achieved through the inclusion of an `API Key` and `base64EncodedAuthorization` together in the header of your HTTP requests.

```
"Authorization": "IYZWSv2"+" "+"base64EncodedAuthorization"
```

Example Request Header:

```
POST /payment/bin/check HTTP/1.1
Host: sandbox-api.iyzipay.com
Authorization: IYZWSv2 YXBpS2V5OnNhbmRib3gtbDlNZDFHajNJWWNtdTROZGFXe
Content-Type: application/json
x-iyzi-rnd: 123456789 // x-iyzi-rnd used to be random key from previ
```

Overview

Here's a breakdown of the required components:

- **apiKey**: Your unique API key assigned to your account.
- **secretKey**: Your secret key associated with your account.
- **x-iyzi-rnd**: A randomly generated number by merchants that included in the request header for each API call. (x-iyzi-rnd used to be random key from previous SHA1 Authentication)
- **encryptedData:** The encrypted version of the request payload parameters with HMACSHA256.

Authentication can be divided into three sequential steps:

1. encryptedData
2. base64Encoded
3. Authorization

1. encryptedData

The `encryptedData` represents a encrypted version of the request payload, the process entails generating a hash using HMACSHA256 encryption.

The signature is generated using the following formula, relatively;

```
HMACSHA256(randomKey + uri.path + request.body, secretKey)
```

`randomKey` ;

- could be either x-iyzi-rnd used to be random key from previous SHA1 Authentication.
- Or please do not hesitate generate randomly.

With in a sample, below you may find a dummy Bin Check request curl;

```
curl
--location --request POST 'https://api.iyzipay.com/payment/bin/checl
--header 'Authorization: IYZWSv2 ***' \
--header 'x-iyzi-rnd: 123456789' \
--header 'Content-Type: application/json' \
--data-raw '{
    "locale":"tr",
    "binNumber":"535805",
    "conversationId": "docsTest-v1"
}'
```

`encryptedData` for that Bin Check request above is;

```
079df4b2426fc7f4208d8f22fbc0349794019f8ce2b0711de7808b4874f4e796
```

2. base64Encoded

Assuming that the `encryptedData` has been generated correctly, it is now time for Base64 encryption.

The signature is generated using the following formula, relatively;

```
base64("apiKey:"+apiKey+"&randomKey:"+randomKey+"&signature:"+encry
```

The result is our `base64EncodedAuthorization` to be used in the header.

3. Authorization

After all the operations, the final and simplest step is to include `IYZWSv2`, `base64EncodedAuthorization` in the header, relatively.

```
...
"Authorization": "IYZWSv2"+" "+"base64EncodedAuthorization"
...
```

In between IYZWSv2 and base64EncodedAuthorization there is a single line space.

Sample Pre-request Script of Authorization on Postman

Taking a dummy Bin Check request as an example, the authorization process would be as follows;

```
var apiKey = environment.apiKey;
var secretKey = environment.secretKey;

//Generate authorization string
function generateAuthorizationString() {

    // Lets create uniq randomKey
    // Sample randomKey : 172224601709012345678 9
    // x-iyzi-rnd used to be random key from previous SHA1 Authentic
    var randomKey = new Date().getTime() + "123456789";

    // Get the uri path for this request
    // Sample uri_path : /payment/bin/check
    var uri_path = "/payment/bin/check";

    // Get the payload and concatanete with uri path and randomKey.
    // Sample payload : payload: 172224601709012345678 9/payment/bin/
    var payload = _.isEmpty(request.data) ? randomKey + uri_path : r

    // Encrypt the payload with HMACSHA256
    // Sample encryptedData : 91e491486d3aa951b4f387cc93d67fc754c472
    var encryptedData = CryptoJS.HmacSHA256(payload, secretKey);

    // Create the authorizationString using encryptedData
    /* Sample authorizationString : apiKey:sandbox-3uHv0LccjcWDyFHTv
                                    randomKey:1722246017090123456
                                    signature:91e491486d3aa951b4
    var authorizationString = "apiKey:" + apiKey
                            + "&randomKey:" + randomKey
                            + "&signature:" + encryptedData;


    // Encode the authorizationString with base64
    // Sample base64EncodedAuthorization : YXBpS2V5OnNhbmRib3gtM3VIc
    var base64EncodedAuthorization = CryptoJS.enc.Base64.stringify(C

    // Concatanate the encoded authorizationString with 'IYZWSv2 '.
    // Sample return value : IYZWSv2 YXBpS2V5OnNhbmRib3gtM3VIdjBMY2M
    return "IYZWSv2 " + base64EncodedAuthorization;
}
var authorization = generateAuthorizationString();
postman.setEnvironmentVariable("authorization", authorization);
```

Previous
Authentication

Next
Postman Collections

Last updated 5 months ago